

Universidad Privada Domingo Savio

Tecnología Web II

Entrega 9B

Consulta de Asistencias: Estudiante, Admin y Estadísticas

FASE 4: Horarios, Asistencias y Configuración de Evaluación

Abril 2026

Objetivos de Aprendizaje

- Implementar API Routes de solo lectura para consulta de asistencias (estudiante y admin).
- Construir un componente 'Mis Asistencias' que muestre al estudiante su historial por materia con porcentaje de asistencia.
- Crear una vista de resumen global de asistencias para el admin con indicadores de riesgo académico.
- Calcular estadísticas agregadas (porcentajes, conteos, promedios) en el frontend a partir de datos brutos.
- Integrar tarjetas de estadísticas de asistencia en los dashboards existentes de admin y docente.

Prerrequisitos

- ✓ Entrega 9A completada: módulo de asistencias funcional (docente puede pasar lista).
- ✓ Al menos 2-3 registros de asistencia creados desde el componente PasarLista.
- ✓ Políticas RLS de asistencias activas (Entrega 8A): estudiante solo ve las suyas, admin ve todas.
- ✓ Componentes de dashboard existentes desde las Entregas 6A-7B.

Teoría: Consulta vs Registro

En la Entrega 9A implementamos el lado de ESCRITURA (docente registra asistencia). Ahora implementamos el lado de LECTURA (estudiante y admin consultan). Esta separación es un patrón clásico en arquitectura de software conocido como CQRS simplificado (Command Query Responsibility Segregation).

Aspecto	Registro (9A)	Consulta (9B)
Quien lo usa	Docente	Estudiante y Admin
Operación HTTP	POST (crear), PUT (editar)	GET (solo lectura)
Complejidad de datos	Entrada: array de estados	Salida: datos agregados + estadísticas
Componente principal	PasarLista (formulario)	MisAsistencias (tabla + stats)
Validación	Zod en entrada	Sin validación (solo lectura)
Riesgo	Datos incorrectos	Performance con muchos registros

Cálculo de Porcentaje de Asistencia

Formula de Porcentaje

$\text{Porcentaje} = (\text{presentes} + \text{tardanzas}) / \text{total_sesiones} * 100$

Donde:

presentes = registros con estado 'presente'
 tardanzas = registros con estado 'tardanza' (cuenta como asistencia)
 total_sesiones = total de registros para esa inscripción

Ejemplo:

20 sesiones totales
 15 presentes + 3 tardanzas + 1 justificado + 1 ausente
 $\text{Porcentaje} = (15 + 3) / 20 * 100 = 90\%$

Umbral UCB (referencia):

>= 80% : Normal (verde)
 70-79% : Alerta (amarillo)
 < 70% : Riesgo (rojo) - posible inhabilitación

NOTA: Tardanza cuenta como asistencia

En el reglamento académico boliviano, la tardanza cuenta como asistencia (el estudiante SI estuvo presente, solo llegó tarde). Por eso sumamos presentes + tardanzas en el numerador. El estado

'justificado' NO suma como asistencia pero tampoco penaliza en algunos reglamentos; aquí lo dejamos fuera del numerador por simplicidad.

Paso 1: API Route Estudiante - GET Asistencias

El estudiante consulta sus asistencias agrupadas por materia. La API devuelve los registros junto con estadísticas pre-calculadas para cada materia inscrita.

```
TypeScript - app/api/estudiante/asistencias/route.ts (crear archivo nuevo)
import { createClient } from "@lib/supabase/server";
import { NextResponse } from "next/server";

export async function GET() {
  const supabase = await createClient();

  const { data: { user }, error: authError } = await supabase.auth.getUser();
  if (authError || !user) {
    return NextResponse.json({ error: "No autenticado" }, { status: 401 });
  }

  // Obtener inscripciones activas del estudiante
  const { data: inscripciones, error: inscError } = await supabase
    .from("inscripciones")
    .select(`
      id,
      materia_id,
      materias (id, nombre, codigo)
    `)
    .eq("estudiante_id", user.id)
    .eq("estado", "inscrito");

  if (inscError) {
    return NextResponse.json({ error: inscError.message }, { status: 500 });
  }

  if (!inscripciones || inscripciones.length === 0) {
    return NextResponse.json({ materias: [] });
  }

  // Obtener todas las asistencias del estudiante
  const inscripcionIds = inscripciones.map((i) => i.id);

  const { data: asistencias, error: asistError } = await supabase
    .from("asistencias")
    .select("*")
    .in("inscripcion_id", inscripcionIds)
    .order("fecha", { ascending: false });

  if (asistError) {
    return NextResponse.json({ error: asistError.message }, { status: 500 });
  }
}
```

```

}

// Agrupar por materia y calcular estadísticas
const materias = inscripciones.map((insc) => {
  const asistMateria = asistencias?.filter(
    (a) => a.inscripcion_id === insc.id
  ) || [];

  const total = asistMateria.length;
  const presentes = asistMateria.filter(
    (a) => a.estado === "presente"
  ).length;
  const tardanzas = asistMateria.filter(
    (a) => a.estado === "tardanza"
  ).length;
  const ausentes = asistMateria.filter(
    (a) => a.estado === "ausente"
  ).length;
  const justificados = asistMateria.filter(
    (a) => a.estado === "justificado"
  ).length;

  const porcentaje = total > 0
    ? Math.round(((presentes + tardanzas) / total) * 100)
    : 0;

  return {
    materia: insc.materias,
    inscripcion_id: insc.id,
    estadisticas: {
      total, presentes, tardanzas, ausentes,
      justificados, porcentaje,
    },
    registros: asistMateria,
  };
});

return NextResponse.json({ materias });
}

```

TIP: Estadísticas calculadas en el servidor

Calculamos los porcentajes en la API Route (no en el frontend) por dos razones: (1) evitamos enviar datos brutos innecesarios al cliente, y (2) la lógica de cálculo queda centralizada. Sin embargo, también enviamos los registros[] para que el estudiante pueda ver el detalle por fecha.

Paso 2: API Route Admin - GET Resumen Global

El admin necesita una vista panorámica: todas las materias con su porcentaje promedio de asistencia y los estudiantes en riesgo (menos de 70%). Esta API agrega datos de TODAS las materias.

TypeScript - app/api/admin/asistencias/route.ts (crear archivo nuevo)

```
import { createClient } from "@lib/supabase/server";
import { NextResponse } from "next/server";

export async function GET() {
  const supabase = await createClient();

  // Verificar admin
  const { data: { user }, error: authError } = await supabase.auth.getUser();
  if (authError || !user) {
    return NextResponse.json({ error: "No autenticado" }, { status: 401 });
  }

  const { data: perfil } = await supabase
    .from("usuarios_perfil")
    .select("rol")
    .eq("id", user.id)
    .single();

  if (perfil?.rol !== "admin") {
    return NextResponse.json({ error: "Acceso denegado" }, { status: 403 });
  }

  // Obtener TODAS las materias con sus inscripciones
  const { data: materias } = await supabase
    .from("materias")
    .select(`
      id, nombre, codigo,
      usuarios_perfil!materias_docente_id_fkey (nombre, apellido),
      inscripciones (id, estudiante_id)
    `)
    .order("nombre");

  if (!materias || materias.length === 0) {
    return NextResponse.json({ resumen: [], global: {} });
  }

  // Obtener TODAS las asistencias
  const { data: todasAsistencias } = await supabase
    .from("asistencias")
    .select("inscripcion_id, estado, fecha");
```

```

const asistenciasMap = new Map<string, any[]>();
(todasAsistencias || []).forEach((a) => {
  const arr = asistenciasMap.get(a.inscripcion_id) || [];
  arr.push(a);
  asistenciasMap.set(a.inscripcion_id, arr);
});

// Calcular resumen por materia
let totalGlobalRegistros = 0;
let totalGlobalPresentes = 0;
let estudiantesEnRiesgo = 0;
let totalEstudiantes = 0;

const resumen = materias.map((mat) => {
  const inscripciones = (mat.inscripciones as any[]) || [];
  let totalMat = 0;
  let presentesMat = 0;
  let enRiesgoMat = 0;

  const detalleEstudiantes = inscripciones.map((insc) => {
    const asist = asistenciasMap.get(insc.id) || [];
    const total = asist.length;
    const presentes = asist.filter(
      (a: any) => a.estado === "presente" || a.estado === "tardanza"
    ).length;
    const pct = total > 0 ? Math.round((presentes / total) * 100) : null;

    totalMat += total;
    presentesMat += presentes;
    if (pct !== null && pct < 70) enRiesgoMat++;

    return { inscripcion_id: insc.id, total, presentes, porcentaje: pct };
  });

  const promedioMateria = totalMat > 0
    ? Math.round((presentesMat / totalMat) * 100)
    : null;

  totalGlobalRegistros += totalMat;
  totalGlobalPresentes += presentesMat;
  estudiantesEnRiesgo += enRiesgoMat;
  totalEstudiantes += inscripciones.length;

  return {
    materia: {
      id: mat.id,
      nombre: mat.nombre,
      codigo: mat.codigo,
      docente: mat.usuarios_perfil,
    },
    resumen,
    promedioMateria,
  };
});

```



```

    },
    total_inscritos: inscripciones.length,
    total_sesiones: totalMat > 0
      ? Math.round(totalMat / Math.max(inscripciones.length, 1))
      : 0,
    promedio_asistencia: promedioMateria,
    estudiantes_en_riesgo: enRiesgoMat,
  });
});

const promedioGlobal = totalGlobalRegistros > 0
  ? Math.round((totalGlobalPresentes / totalGlobalRegistros) * 100)
  : 0;

return NextResponse.json({
  resumen,
  global: {
    total_materias: materias.length,
    total_estudiantes: totalEstudiantes,
    promedio_asistencia: promedioGlobal,
    estudiantes_en_riesgo: estudiantesEnRiesgo,
  },
});
}

```

ADVERTENCIA: Performance con muchos datos

Esta API obtiene TODAS las asistencias en una sola consulta. Para un sistema con 500+ estudiantes y meses de datos, sería conveniente paginar o filtrar por periodo. Para el MVP del diplomado, este enfoque es aceptable porque el volumen es bajo.

Paso 3: Componente Mis Asistencias (Estudiante)

Este componente muestra al estudiante un resumen visual por materia con tarjetas de porcentaje y una tabla detallada expandible para ver cada fecha.

TypeScript - components/dashboard/estudiante/mis-asistencias.tsx (Parte 1)

```
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";
import {
  CheckCircle, XCircle, Clock, AlertTriangle,
  ChevronDown, ChevronUp,
} from "lucide-react";

import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Badge } from "@components/ui/badge";
import { Progress } from "@components/ui/progress";
import { Button } from "@components/ui/button";
import {
  Table, TableBody, TableCell, TableHead,
  TableHeader, TableRow,
} from "@components/ui/table";

interface Estadisticas {
  total: number;
  presentes: number;
  tardanzas: number;
  ausentes: number;
  justificados: number;
  porcentaje: number;
}

interface Registro {
  id: string;
  fecha: string;
  estado: string;
  observaciones: string | null;
}

interface MateriaAsistencia {
  materia: { id: string; nombre: string; codigo: string };
  inscripcion_id: string;
  estadisticas: Estadisticas;
  registros: Registro[];
}
```

TypeScript - mis-asistencias.tsx (Parte 2: Componente Principal)

```

export default function MisAsistencias() {
  const [materias, setMaterias] = useState<MateriaAsistencia[]>([]);
  const [expandida, setExpandida] = useState<string | null>(null);
  const [cargando, setCargando] = useState(true);

  useEffect(() => {
    async function cargar() {
      try {
        const res = await fetch("/api/estudiante/asistencias");
        if (res.ok) {
          const data = await res.json();
          setMaterias(data.materias);
        }
      } catch {
        toast.error("Error al cargar asistencias");
      } finally {
        setCargando(false);
      }
    }
    cargar();
  }, []);

  function colorPorcentaje(pct: number) {
    if (pct >= 80) return "text-green-600";
    if (pct >= 70) return "text-yellow-600";
    return "text-red-600";
  }

  function progressColor(pct: number) {
    if (pct >= 80) return "bg-green-500";
    if (pct >= 70) return "bg-yellow-500";
    return "bg-red-500";
  }

  function badgeEstado(estado: string) {
    const map: Record<string, { variant: string; icon: any }> = {
      presente: { variant: "bg-green-100 text-green-800", icon: CheckCircle },
      ausente: { variant: "bg-red-100 text-red-800", icon: XCircle },
      tardanza: { variant: "bg-orange-100 text-orange-800", icon: Clock },
      justificado: { variant: "bg-yellow-100 text-yellow-800", icon: AlertTriangle },
    };
    const config = map[estado] || map.presente;
    const Icon = config.icon;
    return (
      <Badge className={config.variant}>
        <Icon className="h-3 w-3 mr-1" />
        {estado.charAt(0).toUpperCase() + estado.slice(1)}
      </Badge>
    );
  }
}

```

```

}

if (cargando) {
  return <p className="text-muted-foreground p-6">Cargando...</p>;
}

if (materias.length === 0) {
  return (
    <Card>
      <CardContent className="py-12 text-center">
        <p className="text-muted-foreground">
          No hay registros de asistencia aun.
        </p>
      </CardContent>
    </Card>
  );
}

```

TypeScript - mis-asistencias.tsx (Parte 3: Render)

```

return (
  <div className="space-y-4">
    {materias.map((mat) => (
      <Card key={mat.inscripcion_id}>
        <CardHeader
          className="cursor-pointer"
          onClick={() =>
            setExpandida(
              expandida === mat.inscripcion_id
                ? null
                : mat.inscripcion_id
            )
          }
        </CardHeader>
        <div>
          <CardTitle className="text-lg">
            {mat.materia.codigo} - {mat.materia.nombre}
          </CardTitle>
          <div className="flex gap-4 mt-2 text-sm text-muted-foreground">
            <span>Sesiones: {mat.estadisticas.total}</span>
            <span>Presentes: {mat.estadisticas.presentes}</span>
            <span>Tardanzas: {mat.estadisticas.tardanzas}</span>
            <span>Ausentes: {mat.estadisticas.ausentes}</span>
          </div>
        </div>
      </Card>
    ))}
    <div className="flex items-center justify-between">
      <div>
        <CardTitle className="text-lg">
          {mat.materia.codigo} - {mat.materia.nombre}
        </CardTitle>
        <div className="flex gap-4 mt-2 text-sm text-muted-foreground">
          <span>Sesiones: {mat.estadisticas.total}</span>
          <span>Presentes: {mat.estadisticas.presentes}</span>
          <span>Tardanzas: {mat.estadisticas.tardanzas}</span>
          <span>Ausentes: {mat.estadisticas.ausentes}</span>
        </div>
      </div>
      <div>
        <CardTitle className="text-lg">
          {mat.materia.codigo} - {mat.materia.nombre}
        </CardTitle>
        <div className="flex gap-4 mt-2 text-sm text-muted-foreground">
          <span>Sesiones: {mat.estadisticas.total}</span>
          <span>Presentes: {mat.estadisticas.presentes}</span>
          <span>Tardanzas: {mat.estadisticas.tardanzas}</span>
          <span>Ausentes: {mat.estadisticas.ausentes}</span>
        </div>
      </div>
    </div>
  </div>
);

```

```

        mat.estadisticas.porcentaje
      )}``}>
      {mat.estadisticas.porcentaje}%
    </p>
    <p className="text-xs text-muted-foreground">asistencia</p>
  </div>

  {/* Expand toggle */}
  {expandida === mat.inscripcion_id
    ? <ChevronUp className="h-5 w-5" />
    : <ChevronDown className="h-5 w-5" />
  }
</div>
</div>

{/* Barra de progreso */}
<div className="mt-3">
  <Progress
    value={mat.estadisticas.porcentaje}
    className="h-2"
  />
</div>

{/* Alerta si esta en riesgo */}
{mat.estadisticas.porcentaje < 70 &&
  mat.estadisticas.total > 0 && (
    <div className="mt-2 p-2 bg-red-50 border border-red-200
      rounded-md text-sm text-red-700 flex
      items-center gap-2">
      <AlertTriangle className="h-4 w-4" />
      Atencion: Tu asistencia esta por debajo del 70%.
      Podrias ser inhabilitado segun reglamento.
    </div>
  )}
</CardHeader>

{/* Tabla expandible */}
{expandida === mat.inscripcion_id && (
  <CardContent>
    <Table>
      <TableHeader>
        <TableRow>
          <TableHead>Fecha</TableHead>
          <TableHead>Estado</TableHead>
          <TableHead>Observaciones</TableHead>
        </TableRow>
      </TableHeader>
      <TableBody>
        {mat.registros.map((r) => (
          <TableRow key={r.id}>
            <TableCell className="font-mono text-sm">
              {r.fecha}
            </TableCell>
          </TableRow>
        ))}
      </TableBody>
    </Table>
  </CardContent>
)}

```

```

        </TableCell>
        <TableCell>{badgeEstado(r.estado)}</TableCell>
        <TableCell className="text-sm
            text-muted-foreground">
            {r.observaciones || "-"}
        </TableCell>
    </TableRow>
    )})
</TableBody>
</Table>
</CardContent>
    )}
</Card>
    )})
</div>
);
}

```

TIP: Patron accordion manual

Usamos estado expandida (string | null) para controlar cual Card esta abierta. Al hacer click en el header, se alterna. Solo una materia se expande a la vez. Si necesitaras multiples abiertas simultaneamente, usarias un Set<string> en lugar de un string.

Paso 4: Componente Resumen Global (Admin)

El admin ve tarjetas de resumen global y una tabla con el porcentaje promedio de asistencia por materia, resaltando aquellas con estudiantes en riesgo.

TypeScript - components/dashboard/admin/asistencias-resumen.tsx

```
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";
import {
  Users, BarChart3, AlertTriangle, CheckCircle,
} from "lucide-react";

import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Badge } from "@components/ui/badge";
import { Progress } from "@components/ui/progress";
import {
  Table, TableBody, TableCell, TableHead,
  TableHeader, TableRow,
} from "@components/ui/table";

interface ResumenMateria {
  materia: {
    id: string; nombre: string; codigo: string;
    docente: { nombre: string; apellido: string };
  };
  total_inscritos: number;
  total_sesiones: number;
  promedio_asistencia: number | null;
  estudiantes_en_riesgo: number;
}

interface DatosGlobales {
  total_materias: number;
  total_estudiantes: number;
  promedio_asistencia: number;
  estudiantes_en_riesgo: number;
}

export default function AsistenciasResumen() {
  const [resumen, setResumen] = useState<ResumenMateria[]>([]);
  const [global, setGlobal] = useState<DatosGlobales | null>(null);
  const [cargando, setCargando] = useState(true);

  useEffect(() => {
    async function cargar() {
```

```

    try {
      const res = await fetch("/api/admin/asistencias");
      if (res.ok) {
        const data = await res.json();
        setResumen(data.resumen);
        setGlobal(data.global);
      }
    } catch {
      toast.error("Error al cargar resumen");
    } finally {
      setCargando(false);
    }
  }
  cargar();
}, []);

if (cargando) {
  return <p className="text-muted-foreground">Cargando resumen...</p>;
}

return (
  <div className="space-y-6">
    {/* Tarjetas globales */}
    {global && (
      <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
        <Card>
          <CardContent className="pt-6">
            <div className="flex items-center gap-3">
              <BarChart3 className="h-8 w-8 text-blue-500" />
              <div>
                <p className="text-2xl font-bold">
                  {global.promedio_asistencia}%
                </p>
                <p className="text-sm text-muted-foreground">
                  Asistencia Promedio
                </p>
              </div>
            </div>
          </CardContent>
        </Card>

        <Card>
          <CardContent className="pt-6">
            <div className="flex items-center gap-3">
              <Users className="h-8 w-8 text-green-500" />
              <div>
                <p className="text-2xl font-bold">
                  {global.total_estudiantes}
                </p>
                <p className="text-sm text-muted-foreground">
                  Inscripciones Activas
                </p>
              </div>
            </div>
          </CardContent>
        </Card>
      </div>
    )}
  </div>
);

```



```

        </div>
      </div>
    </CardContent>
  </Card>

  <Card>
    <CardContent className="pt-6">
      <div className="flex items-center gap-3">
        <CheckCircle className="h-8 w-8 text-emerald-500" />
        <div>
          <p className="text-2xl font-bold">
            {global.total_materias}
          </p>
          <p className="text-sm text-muted-foreground">
            Materias Registradas
          </p>
        </div>
      </div>
    </CardContent>
  </Card>

  <Card className={global.estudiantes_en_riesgo > 0
    ? "border-red-200 bg-red-50" : ""}>
  >
    <CardContent className="pt-6">
      <div className="flex items-center gap-3">
        <AlertTriangle className={`h-8 w-8 ${
          global.estudiantes_en_riesgo > 0
            ? "text-red-500" : "text-gray-400"
        }`} />
        <div>
          <p className="text-2xl font-bold">
            {global.estudiantes_en_riesgo}
          </p>
          <p className="text-sm text-muted-foreground">
            Estudiantes en Riesgo
          </p>
        </div>
      </div>
    </CardContent>
  </Card>
</div>
)}}

{/* Tabla por materia */}
<Card>
  <CardHeader>
    <CardTitle>Asistencia por Materia</CardTitle>
  </CardHeader>
  <CardContent>
    <Table>
      <TableHeader>

```

```

<TableRow>
  <TableHead>Materia</TableHead>
  <TableHead>Docente</TableHead>
  <TableHead>Inscritos</TableHead>
  <TableHead>Sesiones</TableHead>
  <TableHead>Promedio</TableHead>
  <TableHead>En Riesgo</TableHead>
</TableRow>
</TableHeader>
<TableBody>
  {resumen.map((r) => (
    <TableRow key={r.materia.id}>
      <TableCell className="font-medium">
        {r.materia.codigo} - {r.materia.nombre}
      </TableCell>
      <TableCell>
        {r.materia.docente?.apellido},
        {" "}{r.materia.docente?.nombre}
      </TableCell>
      <TableCell>{r.total_inscritos}</TableCell>
      <TableCell>{r.total_sesiones}</TableCell>
      <TableCell>
        {r.promedio_asistencia !== null ? (
          <div className="flex items-center gap-2">
            <Progress
              value={r.promedio_asistencia}
              className="w-16 h-2"
            />
            <span className={`font-mono text-sm ${
              r.promedio_asistencia >= 80
                ? "text-green-600"
                : r.promedio_asistencia >= 70
                ? "text-yellow-600"
                : "text-red-600"
            }`}>
              {r.promedio_asistencia}%
            </span>
          </div>
        ) : (
          <span className="text-muted-foreground">
            Sin datos
          </span>
        )}
      </TableCell>
      <TableCell>
        {r.estudiantes_en_riesgo > 0 ? (
          <Badge className="bg-red-100 text-red-800">
            <AlertTriangle className="h-3 w-3 mr-1" />
            {r.estudiantes_en_riesgo}
          </Badge>
        ) : (
          <span className="text-muted-foreground">0</span>
        )}
      </TableCell>
    </TableRow>
  )}

```

```
        </TableCell>
      </TableRow>
    )})
  </TableBody>
</Table>
</CardContent>
</Card>
</div>
);
}
```

IMPORTANTE: Tarjeta de riesgo resaltada

Cuando hay estudiantes en riesgo (menos de 70%), la tarjeta correspondiente se resalta con borde rojo y fondo rosa. Esto permite al admin identificar inmediatamente si hay problemas de asistencia en el sistema.

Paso 5: Actualizar Paginas

TypeScript - app/(dashboard)/estudiante/asistencias/page.tsx (reemplazar)

```
import MisAsistencias from "@components/dashboard/estudiante/mis-asistencias";

export default function AsistenciasEstudiantePage() {
  return (
    <div className="p-6">
      <h1 className="text-2xl font-bold mb-6">Mis Asistencias</h1>
      <MisAsistencias />
    </div>
  );
}
```

TypeScript - app/(dashboard)/admin/asistencias/page.tsx (reemplazar)

```
import AsistenciasResumen from "@components/dashboard/admin/asistencias-resumen";

export default function AsistenciasAdminPage() {
  return (
    <div className="p-6">
      <h1 className="text-2xl font-bold mb-6">
        Resumen de Asistencias
      </h1>
      <AsistenciasResumen />
    </div>
  );
}
```

Paso 6: Tarjeta de Estadísticas para Dashboards

Crea un componente reutilizable que muestre un mini-resumen de asistencia. Se usará en el dashboard principal de admin y docente (las páginas /admin y /docente).

TypeScript - components/dashboard/stats-asistencia-card.tsx (crear archivo nuevo)

```
"use client";

import { useEffect, useState } from "react";
import { Card, CardContent } from "@components/ui/card";
import { UserCheck, AlertTriangle } from "lucide-react";

interface Props {
  apiUrl: string; // "/api/admin/asistencias" o "/api/docente/asistencias?..."
}

export default function StatsAsistenciaCard({ apiUrl }: Props) {
  const [stats, setStats] = useState({
    promedio: 0,
    enRiesgo: 0,
  });

  useEffect(() => {
    async function cargar() {
      try {
        const res = await fetch(apiUrl);
        if (res.ok) {
          const data = await res.json();
          if (data.global) {
            // Formato admin
            setStats({
              promedio: data.global.promedio_asistencia,
              enRiesgo: data.global.estudiantes_en_riesgo,
            });
          } else if (data.materias) {
            // Formato estudiante
            const totales = data.materias.reduce(
              (acc: any, m: any) => ({
                sum: acc.sum + m.estadisticas.porcentaje,
                count: acc.count + 1,
              }),
              { sum: 0, count: 0 }
            );
            setStats({
              promedio: totales.count > 0
                ? Math.round(totales.sum / totales.count)
                : 0,
              enRiesgo: data.materias.filter(
```

```

        (m: any) => m.estadisticas.porcentaje < 70
      ).length,
    });
  }
}
} catch {
  // Silenciar error en stats card
}
}
cargar();
}, [apiUrl]);

return (
  <div className="grid grid-cols-2 gap-4">
    <Card>
      <CardContent className="pt-6 flex items-center gap-3">
        <UserCheck className="h-8 w-8 text-blue-500" />
        <div>
          <p className="text-2xl font-bold">{stats.promedio}%</p>
          <p className="text-sm text-muted-foreground">
            Asistencia Promedio
          </p>
        </div>
      </CardContent>
    </Card>

    <Card className={stats.enRiesgo > 0 ? "border-red-200" : ""}>
      <CardContent className="pt-6 flex items-center gap-3">
        <AlertTriangle className={`h-8 w-8 ${
          stats.enRiesgo > 0 ? "text-red-500" : "text-gray-400"
        }`} />
        <div>
          <p className="text-2xl font-bold">{stats.enRiesgo}</p>
          <p className="text-sm text-muted-foreground">
            En Riesgo (<70%)
          </p>
        </div>
      </CardContent>
    </Card>
  </div>
);
}

```

Uso en los Dashboards Existentes

TypeScript - Agregar en la pagina del dashboard admin

```

// En app/(dashboard)/admin/page.tsx, agregar:
import StatsAsistenciaCard from "@components/dashboard/stats-asistencia-card";

```

```
// Dentro del JSX, despues de las tarjetas existentes:  
<div className="mt-6">  
  <h2 className="text-lg font-semibold mb-3">Asistencias</h2>  
  <StatsAsistenciaCard apiUrl="/api/admin/asistencias" />  
</div>
```

TIP: Componente con apiUrl como prop

StatsAsistenciaCard recibe la URL de la API como prop, no una ruta fija. Esto permite reutilizarlo: el admin pasa /api/admin/asistencias, y en el futuro un docente podría pasar /api/docente/stats con estadísticas de sus materias.

Paso 7: Verificar Componente Progress

El componente Progress de shadcn/ui es necesario para las barras de porcentaje. Si no lo tienes instalado, ejecuta:

```
Bash - Instalar Progress si no existe
npx shadcn@latest add progress
```

También verifica que tengas el componente Tabs (usado en la Entrega 9A):

```
Bash - Instalar Tabs si no existe
npx shadcn@latest add tabs
```

Mapa completo de archivos de esta entrega

Archivo	Tipo	Descripción
app/api/estudiante/asistencias/route.ts	API Route	GET asistencias agrupadas por materia con stats
app/api/admin/asistencias/route.ts	API Route	GET resumen global con estudiantes en riesgo
components/.../estudiante/mis-asistencias.tsx	Componente	Tarjetas expandibles con porcentaje y detalle
components/.../admin/asistencias-resumen.tsx	Componente	4 tarjetas globales + tabla por materia
components/.../stats-asistencia-card.tsx	Componente	Mini-stats reutilizable para dashboards
app/(dashboard)/estudiante/asistencias/page.tsx	Página	Actualizada con MisAsistencias
app/(dashboard)/admin/asistencias/page.tsx	Página	Actualizada con AsistenciasResumen

Verificación Final

- ✓ GET /api/estudiante/asistencias devuelve materias[] con estadísticas y registros por materia.
- ✓ GET /api/admin/asistencias devuelve resumen[] por materia y global{} con totales.
- ✓ MisAsistencias muestra tarjetas con porcentaje, Progress bar coloreada y alerta de riesgo.
- ✓ Las tarjetas son expandibles con click para ver detalle por fecha.
- ✓ AsistenciasResumen muestra 4 tarjetas globales y tabla por materia con docente.
- ✓ La tarjeta 'En Riesgo' se resalta con borde rojo cuando hay estudiantes bajo 70%.
- ✓ StatsAsistenciaCard se renderiza correctamente en el dashboard admin.
- ✓ Progress y Tabs están instalados (npm shadcn@latest add progress tabs).
- ✓ npm run dev funciona sin errores en consola.

Ejercicio Propuesto

1. Tarea 1: Agrega un gráfico de barras (usando Recharts o similar) en MisAsistencias que muestre el porcentaje de asistencia de cada materia visualmente. Pista: recharts ya está disponible como librería en el proyecto.
2. Tarea 2: Implementa un filtro de periodo en el admin (por ejemplo, 'Última semana', 'Último mes', 'Todo') que filtre los datos del resumen. Pista: agrega query params fecha_desde y fecha_hasta a la API.
3. Tarea 3: Crea un endpoint /api/admin/asistencias/riesgo que devuelva solo los estudiantes con porcentaje menor a 70%, con nombre, materia y porcentaje exacto. Usalo para crear una tabla de 'Alerta Temprana' en el dashboard admin.
4. Tarea 4: Agrega un StatsAsistenciaCard al dashboard del estudiante que muestre su promedio general de asistencia y cuántas materias tiene en riesgo.

Proximo: Entrega 10A

En la Entrega 10A implementaremos el módulo de Configuración de Evaluación: API Routes CRUD para definir ponderaciones por materia, validación de suma = 100% a nivel de aplicación, componente de tabla editable con indicador visual de la suma total, y vista de consulta para docentes y estudiantes.

Que vamos a construir	Archivos nuevos
Schema Zod con validacion de suma	lib/validations/configuracion-evaluacion.ts
API Route CRUD admin + docente	app/api/admin/evaluacion/route.ts + [id]/route.ts
API Route lectura docente/estudiante	app/api/docente/evaluacion/route.ts
Componente tabla editable con suma	components/.../config-evaluacion-table.tsx
Componente vista consulta	components/.../ponderaciones-materia.tsx